

Secure File Sharing with S3 and CloudFront

Services: S3, CloudFront, IAM, Lambda@Edge

Description: Implement a secure file-sharing service where users upload files to Amazon S3, and Amazon CloudFront delivers those files securely using signed URLs. Lambda@Edge is used to add an additional security layer by validating incoming requests before granting access.

Skills: CDN setup, secure file access, IAM policies, access control, Lambda@Edge integration

Definition: The Secure File Sharing System is designed to provide a secure and controlled way of accessing files stored in Amazon S3. It uses CloudFront signed URLs to allow temporary access to files and prevents direct public access to the S3 bucket. AWS Lambda@Edge is used to validate requests at the edge location by checking conditions such as custom headers before allowing access. IAM policies are implemented to ensure that only authorized services can interact with the S3 bucket. This system ensures secure file delivery, controlled access, and efficient content distribution using a CDN.

Scope of the Project: The scope of this project is to design and implement a secure file-sharing system that protects files from unauthorized access while enabling controlled and efficient delivery using CloudFront.

This project covers:

- Secure File Storage: Files are stored in S3 with public access completely blocked.
- Controlled Access using Signed URLs: Files can only be accessed using time-limited signed URLs generated securely.
- Content Delivery via CDN: CloudFront distributes files globally with low latency.
- Access Control using IAM: IAM policies restrict access to only authorized users and services.
- Additional Security Layer: Lambda@Edge validates incoming requests (e.g., custom headers) before allowing access.

Methodology:

1. S3 + CloudFront + Signed URLs + Lambda@Edge – Best Method

- Uses all required AWS services: S3, CloudFront, IAM, and Lambda@Edge.
- Provides secure file storage by blocking public access to the S3 bucket.
- Uses CloudFront signed URLs to grant temporary and controlled access to files.
- Lambda@Edge adds an additional security layer by validating requests at the edge location.
- Ensures all file access goes through CloudFront using Origin Access Control (OAC).
- Fully secure, scalable, and suitable for production environments.
- Cost-effective and efficient due to CDN-based content delivery.

2. S3 Public Access – Alternative Method

- Uses S3 bucket with public access enabled to share files directly.
- Easy to implement without additional configuration.
- No authentication or access control mechanisms required.
- Not secure and not suitable for sensitive data.
- No control over who accesses the files or how long they are accessible.

AWS Services used in the Project:

1. Amazon S3 (Simple Storage Service)

Definition: Amazon S3 is an object storage service used to store and retrieve data from the cloud.

Purpose: Provides secure storage for uploaded files.

Role: Stores files privately and blocks public access; allows access only through CloudFront.

2. Amazon CloudFront

Definition: Amazon CloudFront is a Content Delivery Network (CDN) that delivers content with low latency and high performance.

Purpose: Distributes files securely to users.

Role: Serves files from S3 using signed URLs and restricts direct access to the bucket.

3. AWS IAM (Identity and Access Management)

Definition: IAM is a service used to manage access and permissions for AWS resources.

Purpose: Controls secure access to S3 and CloudFront.

Role: Defines policies that allow only authorized access (e.g., s3:GetObject) to S3 objects.

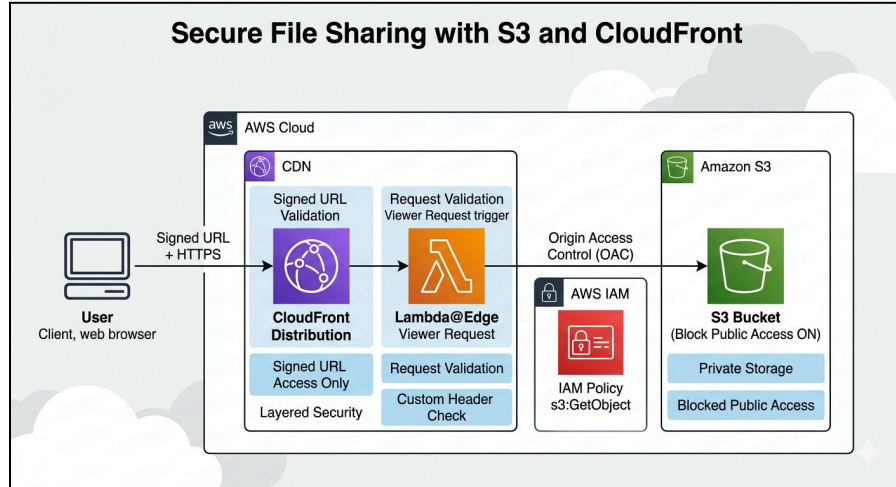
4. AWS Lambda@Edge

Definition: Lambda@Edge is a serverless service that runs code at CloudFront edge locations.

Purpose: Adds additional request validation and security checks.

Role: Validates incoming requests (e.g., headers, tokens) and blocks unauthorized access before reaching S3.

System Architecture Overview



The architecture follows a secure and CDN-based delivery model:

- **Amazon S3:** Stores files securely with no public access
- **Amazon CloudFront:** Delivers content using a CDN
- **Signed URLs:** Provides temporary access to files
- **Lambda@Edge:** Adds an extra layer of request validation
- **AWS IAM:** Manages permissions and access control

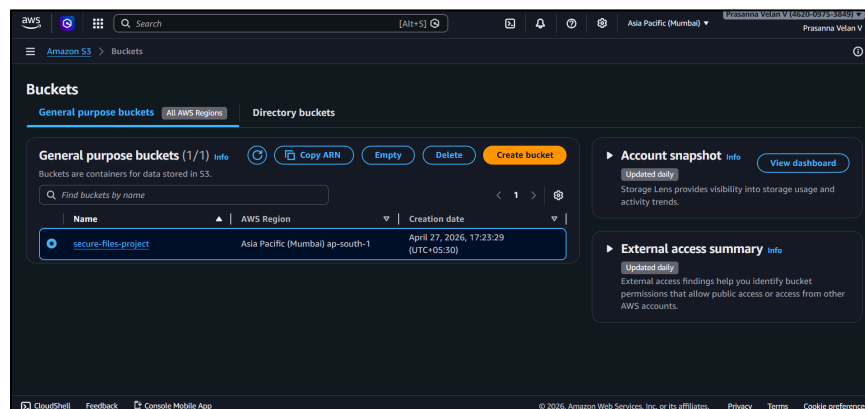
Implementation Steps to Do the Project:

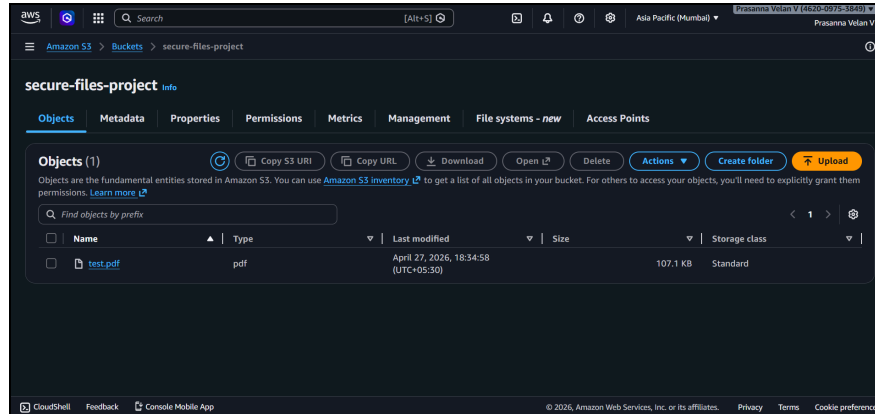
Step 1 :Amazon S3

Amazon S3 is used as the primary storage service. It securely stores files and blocks all public access.

Responsibilities:

- Store uploaded files and ensure private access
- Allow access only through CloudFront





Step 2 :Access Control Configuration

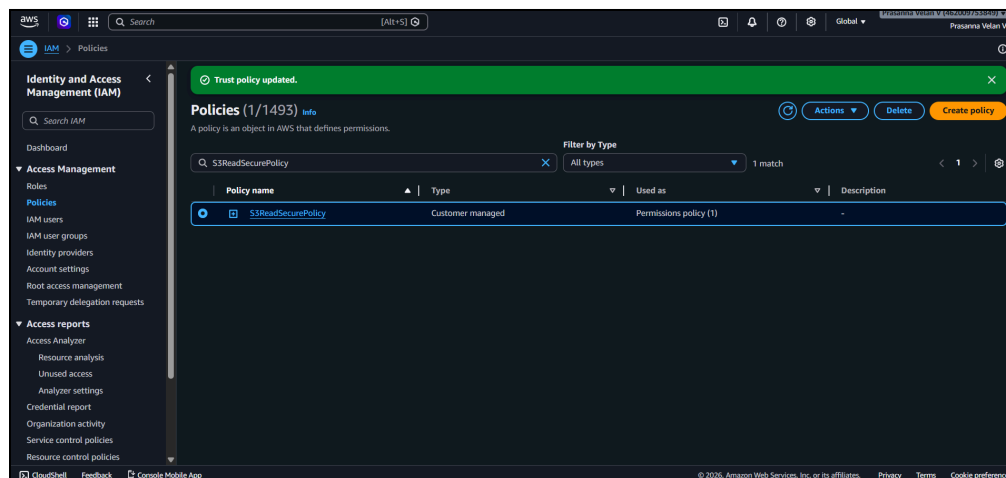
In this step, AWS Identity and Access Management (IAM) is used to define secure and controlled access to the S3 bucket. Instead of allowing open access, specific permissions are granted through IAM policies to ensure that only authorized entities can access the stored files.

IAM policies are created to allow limited actions such as read-only access (s3:GetObject) on the S3 bucket. A dedicated IAM user or role is then configured and attached with this policy, following the principle of least privilege.

Key Points:

- Only required permissions (e.g., read access) are granted to the IAM user/role
- Prevents unauthorized users from accessing S3 objects
- Ensures secure interaction between CloudFront and S3
- Helps maintain strict access control and improves overall security

This step is critical because it ensures that access to files is tightly controlled and only allowed through authorized services like CloudFront.

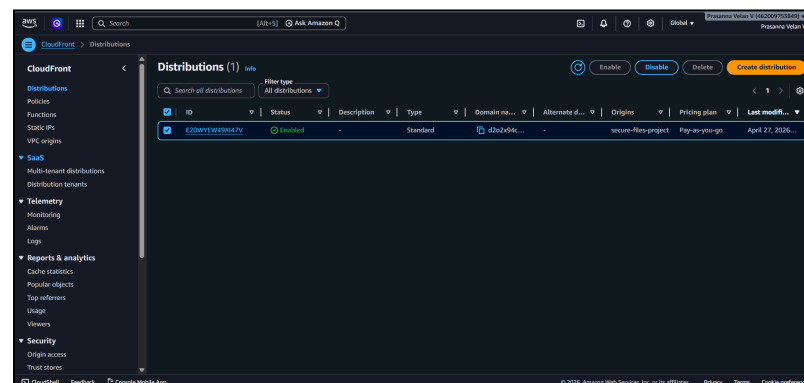
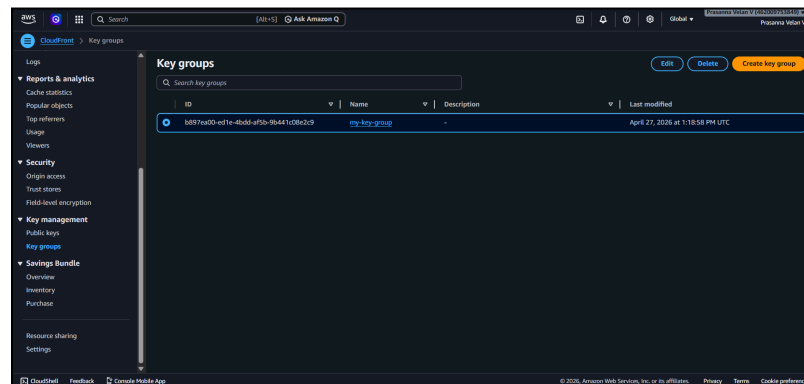


Step 3: Amazon CloudFront

Amazon CloudFront is a Content Delivery Network (CDN) that delivers content to users with low latency by using a global network of edge locations. Instead of directly accessing files from S3, users request content through CloudFront, which improves performance and adds a layer of security.

Key Role:

- Serves content from nearby edge locations for faster access
- Enforces secure access using signed URLs
- Restricts direct access to S3 by using Origin Access Control (OAC)
- Ensures all requests go through CloudFront for validation and control



Step 4: Enable Signed URLs (Core Security Layer)

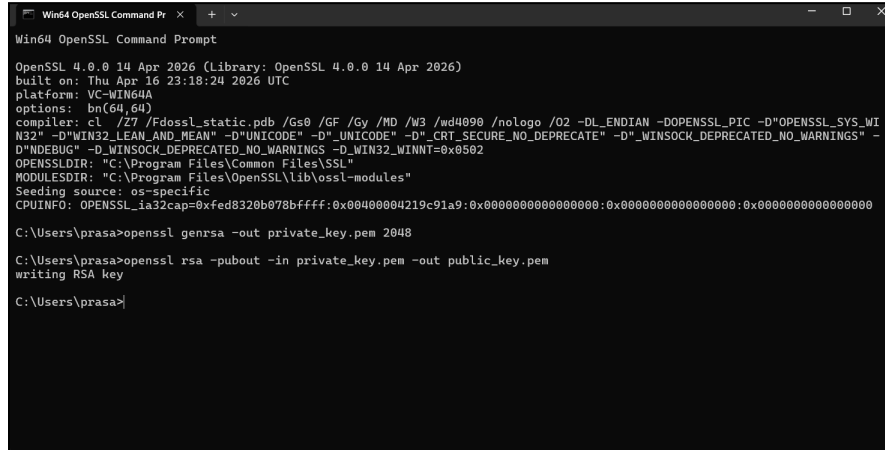
This is the most important step in the project, as it ensures that files are not publicly accessible and can only be accessed through secure, time-limited URLs.

In CloudFront, a **Key Group** is created to manage public keys used for verifying signed URLs. A public-private key pair is generated locally using tools like OpenSSL. The public key is uploaded to CloudFront, while the private key is securely stored and used to generate signed URLs.

When a user requests access to a file, CloudFront checks the signed URL using the public key. If the signature is valid and not expired, access is granted; otherwise, the request is denied.

Key Points:

- Public key is uploaded to CloudFront
- Private key is used to generate signed URLs
- Access is granted only if the URL is valid and not expired
- Prevents direct or unauthorized access to S3 content



```

Win64 OpenSSL Command Prompt
Win64 OpenSSL Command Prompt

OpenSSL 4.0.0 14 Apr 2026 (Library: OpenSSL 4.0.0 14 Apr 2026)
built on: Thu Apr 16 23:18:24 2026 UTC
platform: VC-WIN64A
options: bn(64,64)
compiler: cl /Z7 /Fdssl_static.pdb /Gs0 /GF /Gy /MD /W3 /wd4090 /nologo /O2 -DL_ENDIAN -DOPENSSL_PIC -D"OPENSSL_SYS_WI
N32" -D"WIN32_LEAN_AND_MEAN" -D"UNICODE" -D" _UNICODE" -D" CRT_SECURE_NO_DEPRECATED" -D" _WINSOCK_DEPRECATED_NO_WARNINGS" -
D"NDEBUG" -D_WINSOCK_DEPRECATED_NO_WARNINGS -D_WIN32_WINNT=0x0502
OPENSSLDIR: "C:\Program Files\Common Files\SSL"
MODULESDIR: "C:\Program Files\OpenSSL\lib\openssl-modules"
Seeding source: os-specific
CPUINFO: OPENSSL_ia32cap=0xfed8320b078bffff:0x004000004219c91a9:0x0000000000000000:0x0000000000000000:0x0000000000000000

C:\Users\prasa>openssl genrsa -out private_key.pem 2048

C:\Users\prasa>openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key

C:\Users\prasa>

```

Step 5: Generate Signed URL and Testing

After configuring signed URLs, a secure URL is generated using the private key. This signed URL includes an expiration time, ensuring that access is temporary and controlled.

The signed URL can be generated programmatically (e.g., using Python with AWS SDK). Once generated, it is shared with users to access the file securely.

Testing Behavior:

- Accessing the file using a signed URL →  File is downloaded successfully
- Accessing the file using a normal CloudFront URL →  Access is denied

Key Points:

- Signed URLs are time-bound and expire automatically
- Only users with valid signed URLs can access files
- Ensures secure and controlled file sharing

```

1 import datetime
2 from botocore.signers import CloudFrontSigner
3 from cryptography.hazmat.primitives import hashes
4 from cryptography.hazmat.primitives.asymmetric import padding
5 from cryptography.hazmat.primitives import serialization
6
7 # Load private key
8 with open("private_key.pem", "rb") as key_file:
9     private_key = serialization.load_pem_private_key(
10         key_file.read(),
11         password=None
12     )
13
14 # Signer function
15 def rsa_signer(message):
16     return private_key.sign(
17         message,
18         padding.PKCS1v15(),
19         hashes.SHA1()
20     )
21
22 # FIX THIS

```

Terminal Output:

```

C:\Users\prasa\OneDrive\Desktop\python\generate_url.py:32: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
date_less_than=datetime.datetime.utcnow() + datetime.timedelta(minutes=10)
https://d33kgljg2akq.cloudfront.net/test.pdf?Expires=1772266838&Signature=H0J8L7nGv6ATBYSvM4tx1xG9gh1iMEyJyDQlg-FzVpHdm7pLl1sR2n-zt-ve9Brant-h8nswg9pck3jehf7m0TtXa36-623al9wC-R0F430L0Uc11Tt6HfpeWskotPfeYtx11KCCdK7X1VhoqC-3S2d89P4dK0CqJdYQK5x64j3ysvC6AfguJ81VlesdAg-S2qJW6dbYF6VQMB2Lmoa1QITFSyrVnz411619szV5XGzzK1jrvsWQ9c332QBA1e7J17mVvRk4vX1V-8Lx0jkyf758R13zyQE-qvBcm0xbom4qJKQc2DL0w_&Key-Pair-Id=K3CR36BPM8KU7F
PS C:\Users\prasa\OneDrive\Desktop\python> python generate_url.py
C:\Users\prasa\OneDrive\Desktop\python\generate_url.py:32: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
date_less_than=datetime.datetime.utcnow() + datetime.timedelta(minutes=10)
https://d33kgljg2akq.cloudfront.net/test.pdf?Expires=1772266838&Signature=RbfirQcsbhz8e9pUcaahy2H6w6CgJtDMC3yzYHj0Kc19MQL78LMDT-MH0HC666mo4h10c3Qc2C583T3twdV3--oP58P8TVM0H0bjuV6spC0X0ar3xctubYkBTjhsZ1T7MvYxFC1yAhQCC-nR2so-X-w4.48rvv1U4p3ZxLk2-dkAQ1ZqJlU2d3Mc6F70L6038K2v9m0uT1akg-yu6r00R4Lj41Cubd4e1Jnory670510QTFxwDNTD3app2abxot1cf114M-UstauTE3gToYv233RD-dZZ3Qp-dK0csm80L3UH64r8N1Z04qT3KAp0Pjg_&Key-Pair-Id=K2FUMUF3wB159R

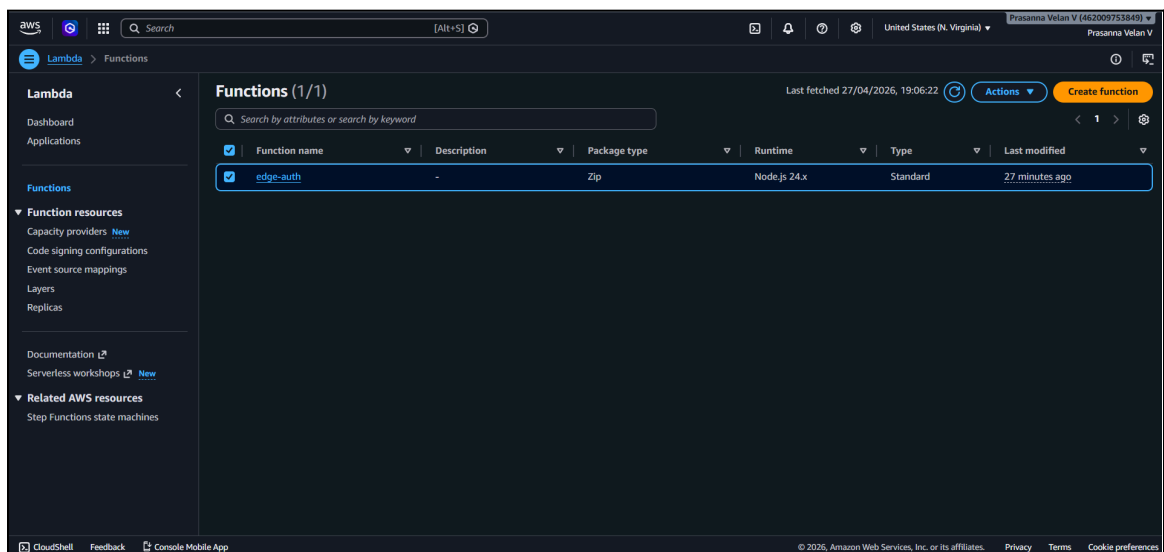
```

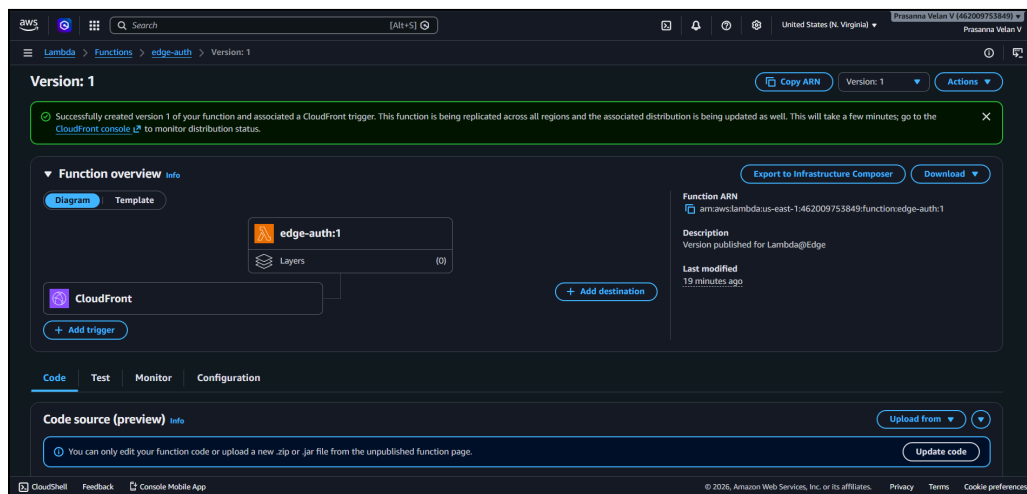
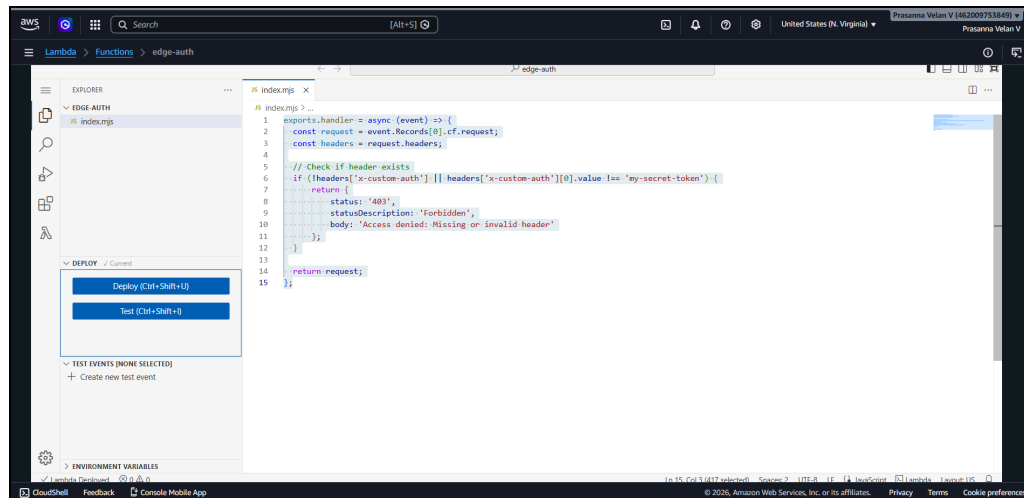
Step 6: AWS Lambda@Edge

AWS Lambda@Edge allows you to run custom code at CloudFront edge locations, closer to the user. This helps in inspecting and modifying requests before they reach the origin (S3), adding an extra layer of security and control.

Responsibilities:

- Validates incoming user requests before allowing access
- Checks for required custom headers, tokens, or conditions
- Blocks unauthorized or invalid requests with appropriate responses
- Enhances security by applying logic at the edge (before reaching S3)





Step 7: Security Strategy

The system uses multiple layers of security:

- Private S3 Bucket (no public access)
- CloudFront OAC to restrict direct access
- Signed URLs for temporary access and Lambda@Edge validation for request filtering

Benefits:

- Prevents unauthorized access
- Ensures controlled file sharing
- Enhances overall system security

503 ERROR

The request could not be satisfied.

The Lambda function associated with the CloudFront distribution is invalid or doesn't have the required permissions. We can't connect to the server for this app or website at this time. There might be too much traffic or a configuration error. Try again later, or contact the app or website owner.
If you provide content to customers through CloudFront, you can find steps to troubleshoot and help prevent this error by reviewing the CloudFront documentation.

Generated by CloudFront (CloudFront)
Request ID: 1W2K3mYU74qM03bhkklj3pExZT60nJo3Y19Yhh6m7y2nW50T7X6g==

Step 8:Access Control Mechanism

The project implements a time-based access mechanism:

- Signed URLs expire after a defined time (e.g., 1 hour)
- Only users with valid URLs can access files
- Additional validation using headers increases security

```
Win64 OpenSSL Command Prompt
Win64 OpenSSL Command Prompt
OpenSSL 4.0.0 14 Apr 2026 (Library: OpenSSL 4.0.0 14 Apr 2026)
built on: Thu Apr 16 23:18:24 2026 UTC
platform: VC-WIN64A
options: bn(64,64)
compiler: cl /Z7 /Fdssl_static.pdb /Gs0 /GF /Gy /MD /W3 /wd4090 /nologo /O2 -DL_ENDIAN -DOPENSSL_PIC -D"OPENSSL_SYS_WI
N32" -D"WIN32_LEAN_AND_MEAN" -D"UNICODE" -D" UNICOD" -D" _CRT_SECURE_NO_DEPRECATED" -D" _WINSOCK_DEPRECATED_NO_WARNINGS" -
D"INDEBU" -D _WINSOCK_DEPRECATED_NO_WARNINGS -D _WIN32_WINNT=0x0502
OPENSSLDIR: "C:\Program Files\Common Files\SSL"
MODULESDIR: "C:\Program Files\OpenSSL\lib\openssl-modules"
Seeding source: os-specific
CPUINFO: OPENSSL_ia32cap=0xfed8320b078bffff:0x00400004219c91a9:0x0000000000000000:0x0000000000000000:0x0000000000000000
C:\Users\prasa>openssl genrsa -out private_key.pem 2048
C:\Users\prasa>openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key
C:\Users\prasa>curl -H "x-custom-auth: my-secret-token" "https://d2o2x94ckzgrxj.cloudfront.net/test.pdf?Expires=17772988
64&Signature=QfWBS7XJ5x1A-SvsapjybcXayWAFRBAZ8iAHkz30C-aV9bkh5igERb97~R7DQ0dsgGCEM9gINkv08By4MPcA9nJ5s0Dry8yNMX-GIAOIx55
Ebmav9qsabnYq87FPCbHY7Ppdncu18Ja00Bh0cQF791CtCSCTDNye3z-IkYRGqGfWyd7PJV0N0bcPS2L5sSUVjLWD7F1DT5zpM3oyuit15AjyuVm9R09KYoQ
igqIkzPuVazCuEXEzKiUjZgYcmShjauLSly7ZK2RV8K9ychaTPzx0mVISUuqED1waEDSulCbhaDAXvds-Mwz9c3qYkVVhwr-eMCWuE0rk-WEilvqVQ__&Ke
y-Pair-Id=K2FUMUF3WNB59R" --output test.pdf
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 1019 100 1019 0 0 1483 0 0 0 0
C:\Users\prasa>
```

Troubleshooting:

Issue 1: Access Denied from CloudFront

- **Cause:** Incorrect bucket policy or OAC not configured
- **Solution:** Ensure CloudFront has access to S3 and Apply correct bucket policy

Issue 2: Signed URL Not Working

- **Cause:** Incorrect key pair or expired URL
- **Solution:** Regenerate signed URL and Verify key pair configuration

Issue 3: Lambda@Edge Not Triggering

- **Cause:** Incorrect region or not deployed properly
- **Solution:** Deploy in us-east-1 and Attach to correct CloudFront event

Issue 4: Access Blocked Even with Valid Request

- **Cause:** Missing custom header or incorrect validation logic
- **Solution:** Check Lambda@Edge code and Ensure required headers are included

Error Handling & Reliability

The system handles failures effectively:

- Invalid or expired signed URLs are rejected
- Unauthorized requests are blocked by Lambda@Edge
- CloudFront ensures high availability and reliability
- Logging and debugging can be done using CloudWatch

Future Enhancements

- Integration with user authentication systems (e.g., Cognito)
- Dynamic file upload portal for users
- Logging and analytics for access tracking
- Expiring download links with advanced policies
- Multi-region replication for high availability

Conclusion

This project demonstrates a secure and scalable approach to file sharing using AWS services. By combining Amazon S3, CloudFront, IAM, and Lambda@Edge, the system ensures that files are not publicly accessible and can only be accessed through controlled mechanisms such as signed URLs.

The solution provides strong security, efficient content delivery, and flexibility for customization. It is suitable for real-world applications where secure file distribution is required, such as document sharing platforms, media delivery systems, and enterprise data access solutions.